

AERIS CHECKPOINT 9 — CONFIG-DRIVEN DYNAMICS INPUTS + TRIM FOUNDATION COMPLETE

Use this as the starting prompt for the next chat.

CONTEXT — WHAT AERIS IS

AERIS is a modular aircraft design software platform, not a collection of scripts.

Its architecture is intentionally layered:

- framework layer
- implementation layer
- pipeline / orchestration layer
- CLI layer

The system is being built as a deterministic, traceable engineering pipeline for:

- parametric geometry generation
- aerodynamic analysis
- structured artifacts
- dataset generation
- future stability / control workflows
- future surrogate / MDAO workflows
- future certification-aware traceability

The philosophy is:

- no script spaghetti
- modular interfaces
- reproducible outputs
- CLI-driven workflows
- explicit artifacts at each stage
- future extensibility without hacks

CURRENT STATE — WHAT IS NOW COMPLETE

1. Geometry system

The native BWB generator is operational.

It supports:

- deterministic geometry generation
- structured planform / section generation
- AeroSandbox geometry reconstruction
- CSV artifact export
- geometry summary JSON export

The geometry summary has been enriched using AeroSandbox wing metadata.

New rich geometry metadata now includes:

- reference values:
 - span
 - area
 - aspect ratio
 - mean geometric chord
 - mean aerodynamic chord
 - taper ratio
 - volume
- mean angles:
 - twist
 - leading-edge sweep
 - quarter-chord sweep
 - trailing-edge sweep
 - quarter-chord dihedral
- aerodynamic center
- sectional metrics:
 - section spans
 - section areas
 - sectional AC coordinates
- geometry info:
 - wing name
 - number of xsecs
 - symmetry
- reference conventions:
 - geometry axes origin convention
 - moment reference point $[0, 0, 0]$
 - x-axis positive aft
 - definitions of reference area / span / chord

This means geometry reference quantities are now explicit and traceable.

2. Aero system

The aerodynamic subsystem is operational and structured.

Capabilities include:

- geometry → AVL pipeline
- flight-condition definition:
 - alpha
 - beta
 - p, q, r
 - velocity
 - altitude
- AVL execution through AeroSandbox
- structured parsing into `aero_result.json`
- support for:
 - force / moment coefficients
 - body-axis derivatives
 - stability-axis derivatives
 - rate derivatives
 - neutral point (Xnp)
 - spiral metric
- sweep workflows and inspection tools

Important reality:

AERIS aero computes linearized aerodynamic information at operating points. It is not a flight simulator.

3. Mass / CG / dynamics foundation

This phase is now operational.

Implemented:

- mass properties model
- inertia placeholders / structured storage
- dynamics foundation artifact

- static margin computation
- consistency check between static margin sign and Cma
- state-space readiness flags
- traceable `dynamics_foundation.json`

The dynamics layer now correctly consumes:

- Xnp from aero
- MAC from geometry summary fallback
- user-defined or config-defined mass / CG / inertia

This bridge was critical because `aero_result.json` does not yet directly store MAC.

4. Dynamics CLI

The following commands now exist and work:

- `aeris dynamics build`
- `aeris dynamics inspect`
- `aeris dynamics cg-sweep`
- `aeris dynamics cg-sweep-inspect`
- `aeris dynamics trim`
- `aeris dynamics trim-inspect`

These are all working from real runs, not just tests.

5. CG sweep capability

AERIS now supports CG sweep over a specified range.

Outputs include:

- `cg_sweep.json`
- `cg_sweep.csv`
- stable CG min / max
- static-margin zero-crossing estimate

Observed real behavior for the current baseline case:

- zero crossing estimate ≈ 0.557037 m
- stable CG range in tested sweep ≈ 0.35 m to 0.55 m
- static margin transitions from positive to negative across that boundary

This is now a usable stability-envelope capability.

6. Config-driven dynamics inputs

Mass / CG / inertia are no longer CLI-only.

Implemented:

- `src/aeris/dynamics/config.py`
- config loader for YAML / JSON mass-properties files
- baseline config file:
 - `configs/mass/baseline_uav.yaml`

CLI behavior now supports:

- pure config-driven build:
 - `aeris dynamics build --run-dir ... --mass-config ...`
- config + CLI override precedence
- config-driven CG sweep

Confirmed behavior:

CLI overrides correctly win over config values.

7. Trim foundation

A first trim layer now exists.

This is **not** full nonlinear trim and **not** control-surface trim.

Current implementation is:

- longitudinal
- control-fixed
- linearized
- based on current C_m , $C_{m\alpha}$, and current α

The implemented idea is:

- estimate $\Delta \alpha_{\text{trim}} = -C_m / C_{m\alpha}$
- compute $\alpha_{\text{trim}} = \alpha_{\text{current}} + \Delta \alpha$

Outputs go to:

- `trim_result.json`

Commands:

- `aeris dynamics trim`
- `aeris dynamics trim-inspect`

Real observed example:

For one baseline run:

- `current alpha = 4.0 deg`
- `current Cm = -0.478110`
- `Cma = -2.517702`
- `estimated trim alpha ≈ -6.88 deg`

This should currently be treated as:

- a useful first-order diagnostic
 - not a final nonlinear truth
 - not yet including control-surface trim
-

VALIDATION STATUS

The following have been validated through real runs and tests:

- generator tests pass
- aero tests pass
- command tests pass
- dynamics tests pass
- config loader tests pass
- build-from-config tests pass
- trim tests pass
- CG sweep tests pass
- CG sweep CSV export tests pass
- CG sweep inspect tests pass

Real workflows have also been executed successfully for:

- `aero run`
- `dynamics build`
- `dynamics inspect`
- `dynamics build from mass config`
- `dynamics build from mass config with CLI override`
- `dynamics cg-sweep from config`

- dynamics cg-sweep-inspect
 - dynamics trim
 - dynamics trim-inspect
-

IMPORTANT CURRENT CONVENTIONS

These conventions are now assumed and should remain explicit unless intentionally changed:

Geometry / moment reference

- airplane reference point currently uses $[0, 0, 0]$
- geometry coordinates are built in that airplane geometry frame
- x-axis is treated as positive aft in dynamics interpretation

Reference quantities

These should vary with geometry, not remain fixed across parametric designs:

- $S_{ref} = \text{wing.area}()$
- $b_{ref} = \text{wing.span}()$
- $c_{ref} = \text{wing.mean_aerodynamic_chord}()$

This is the correct normalization strategy for design-space exploration.

Static margin

Current convention:

- positive static margin means CG is ahead of NP in the positive-aft convention

This convention was validated by CG sweep behavior and C_{ma} consistency checks.

WHAT IS STILL NOT IMPLEMENTED

The following are intentionally not yet complete:

- full nonlinear trim
- control-surface trim
- trim including elevator / elevon effectiveness

- full state-space A/B matrix construction
- eigenvalue / mode analysis
- modal identification
- time response simulation
- control design

These are the next major dynamics steps.

NEXT STEP — WHAT THE NEXT CHAT SHOULD DO

The next step should be:

STEP 11 — STATE-SPACE FOUNDATION

Goal:

Create a clean state-space preparation layer that assembles the required inputs for longitudinal and lateral-directional state-space models.

This should **not** yet jump directly to eigenvalues unless the preparation layer is done correctly first.

Required work

Implement:

- `src/aeris/dynamics/state_space.py`
- models for:
 - `LongitudinalStateSpacePreparation`
 - `LateralDirectionalStateSpacePreparation`
 - `StateSpaceAssemblyResult`
- logic that consumes:
 - `aero_result.json`
 - `dynamics_foundation.json`
- extraction of:
 - operating-point speed
 - mass
 - inertia
 - reference chord / span if needed
 - longitudinal derivatives:
 - `CXu, CXw, CZu, CZw, Cmu, Cmw, Cmq`

- lateral-directional derivatives:
 - CYv, CYp, CYr, Clv, Clp, Clr, Cnv, Cnp, Cnr
- readiness flags and missing-items logic
- artifact:
 - state_space_preparation.json
- CLI:
 - aeris dynamics state-space-build
 - aeris dynamics state-space-inspect

Important constraint

Do not jump straight into eigenvalues until the assembly layer is explicit, testable, and traceable.

DESIGN MINDSET FOR THE NEXT CHAT

The next chat must preserve the current AERIS philosophy:

- modular architecture
- no hacks
- clean CLI
- traceable JSON artifacts
- config-driven reproducibility
- future MDAO / ML compatibility
- future certification-aware reasoning

The next step is **not** about adding flashy features.

It is about building the correct state-space preparation layer that later supports proper modal analysis.